

Ethereum & SmartContracts

Lab 4

Tarea 1

Tarea

Objetivo:

Crear un conjunto de wallets utilizando MetaMask, cargar credito utilizando un faucet, y transferir crédito entre ellas.

Pasos

- Descargar la extensión MetaMask para chrome
- Activar el soporte de testnet (ver presentación de clase)
- Crear dos wallets sobre la red
- Cargar credito a una de ellas utilizando una faucet
- Transferir
- Registrar las TX ids o links al block explorer en la entrega.

Preguntas

- Cuando enviamos una transacción, ¿qué son los parámetros de fee y gas?
- Una vez confirmada, acceder al block explorer y responder:
 - ¿Qué es el Burnt Fee?
 - ¿Qué es el Txn Savings Fee?
 - ¿Por qué no hay datos en el campo datos?

Tarea 2

Tarea

Objetivo:

Desplegar un contrato sencillo en la testnet e interactuar con él

Pasos

- Acceder a <https://remix.ethereum.org/>
- Conectar Remix con MetaMask
- Crear un archivo y utilizar el contrato compartido en clase

```
pragma solidity 0.6.4;

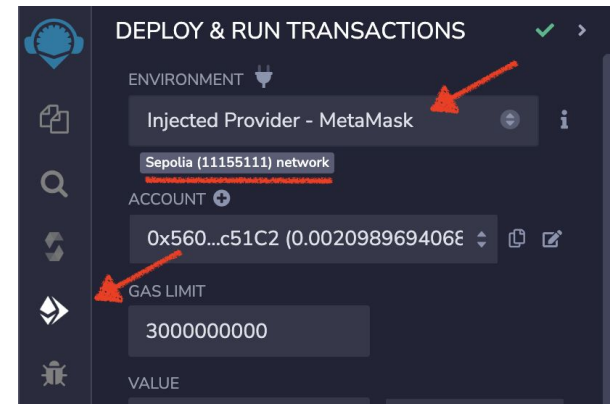
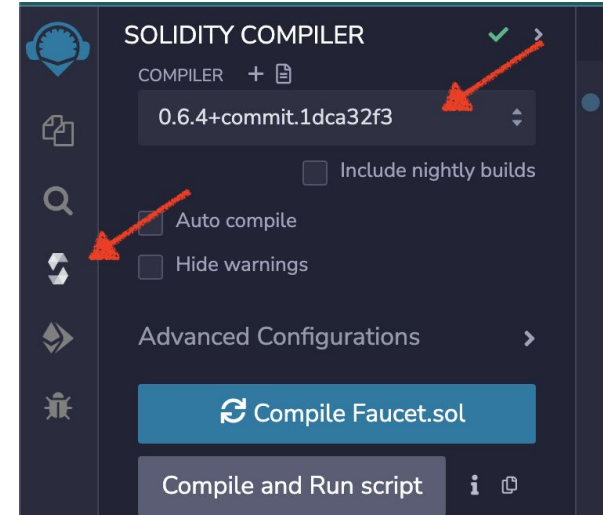
// Our first contract is a faucet!
contract Faucet {
    // Accept any incoming amount
    receive() external payable {}

    // Give out ether to anyone who asks
    function withdraw(uint withdraw_amount) public {
        // Limit withdrawal amount
        require(withdraw_amount <= 1000000000000000000);

        // Send the amount to the address that requested it
        msg.sender.transfer(withdraw_amount);
    }
}
```

Pasos

- Compilar el contrato utilizando el compilador adecuado
- Desplegar el contrato en la testnet utilizando como proveedor MetaMask



Pasos

- Transferir credito al contrato
- Retirar dinero del contrato

Preguntas

- ¿Para qué sirve el gas en este caso?
- Mostrar la historia de la actividad del contrato en el block explorer.

Tarea 3

Tarea

Objetivo:

Analizar un contrato más complejo

Pasos

- A partir de una versión mejorada del contrato anterior, analizarlo y responder?

```
pragma solidity 0.6.4;

contract ContractA {
    address payable public balanced_contract_addr;
    address payable owner;

    constructor() public payable {
        owner = msg.sender;
    }

    receive() external payable {
        if (balanced_contract_addr!= address(0) && msg.sender!=balanced_contract_addr) {
            (bool success, ) = balanced_contract_addr.call{value:msg.value/2}("");
            require(success, "Transfer failed.");
        }
    }

    function setMirror(address payable addr) public {
        require(msg.sender==owner);
        balanced_contract_addr=addr;
    }

    function getBalance() public view returns (uint256) {
        return address(this).balance;
    }

    function withdraw(uint withdraw_amount) public {
        require(withdraw_amount <= 1000000000000000000);
        msg.sender.transfer(withdraw_amount);
    }
}
```

Preguntas

- ¿Qué extensiones tiene el nuevo contrato?
- ¿Cómo desplegaría y conectaría dos contratos?
- ¿Cuál sería el comportamiento?